

BoardVision

Geração de dataset sintético no Blender e treinamento de um detector de peças de jogos

Fernanda Faria Diniz

FastCamp — Dados Sintéticos | Agosto de 2026



240 imagens	720 anotações	99,5% mAP@50	97,7% precisão
-----------------------	-------------------------	------------------------	--------------------------

Este relatório descreve o desenvolvimento de um pipeline completo para detecção de objetos, desde a criação procedural e a randomização da cena no Blender até a anotação automática, o treinamento do YOLOv8n e a avaliação em um conjunto sintético de teste.

1. Introdução

Modelos de visão computacional dependem de imagens representativas e anotações confiáveis. Em dados reais, obter variedade suficiente e desenhar manualmente caixas em cada imagem pode ser demorado. A geração sintética permite controlar a cena e usar a própria geometria 3D para criar as anotações ao mesmo tempo que as imagens são renderizadas.

O projeto final recebeu o nome BoardVision e integra os conhecimentos desenvolvidos ao longo do curso: preparação de cenas no Blender, uso da API Python bpy, randomização de domínio, organização de datasets, anotações no formato YOLO, treinamento por transferência de aprendizado e análise de métricas.

2. Definição do problema e escopo

A tarefa escolhida foi a detecção de objetos. Dada uma imagem de uma superfície de jogo, o modelo deve informar onde cada peça aparece e classificá-la como dado, peão ou ficha. O escopo foi definido para ser simples e permitir a execução completa do pipeline em um computador pessoal, sem depender de ativos externos.

ID	Classe	Construção no Blender	Características visuais
0	dado	cubo chanfrado e cinco marcações	volume alto e face quadrada
1	peão	base, corpo cônico e esfera	silhueta vertical
2	ficha	cilindro baixo e aro superior	silhueta circular e baixa

Variações planejadas

Cada imagem apresenta os três objetos. O script varia a posição no plano, a rotação em torno do eixo vertical, a escala, as cores e a rugosidade dos materiais. Também são modificados o fundo, a posição e o enquadramento da câmera, a energia e a cor de duas luzes de área e a direção de uma luz solar. Uma distância mínima reduz sobreposições extremas, mas ainda permite cenas com objetos próximos.



3. Preparação da cena no Blender

Os três modelos foram construídos proceduralmente pelo próprio script. Essa escolha deixa o projeto independente de downloads e permite reconstruir a cena desde uma instalação limpa do Blender. O dado usa um cubo com bordas chanfradas e cinco pequenos cilindros escuros na face superior; o peão combina uma base cilíndrica, um corpo cônico e uma esfera; a ficha combina um disco com um aro.

A cena contém um plano de apoio, uma câmera ortográfica inclinada, duas luzes de área e uma luz do tipo Sun. O renderizador utilizado foi o Eevee, escolhido por oferecer boa velocidade para um dataset desse porte. Sombras de contato, oclusão ambiente e materiais com diferentes níveis de rugosidade ajudam a produzir pistas visuais sem tornar a renderização pesada.

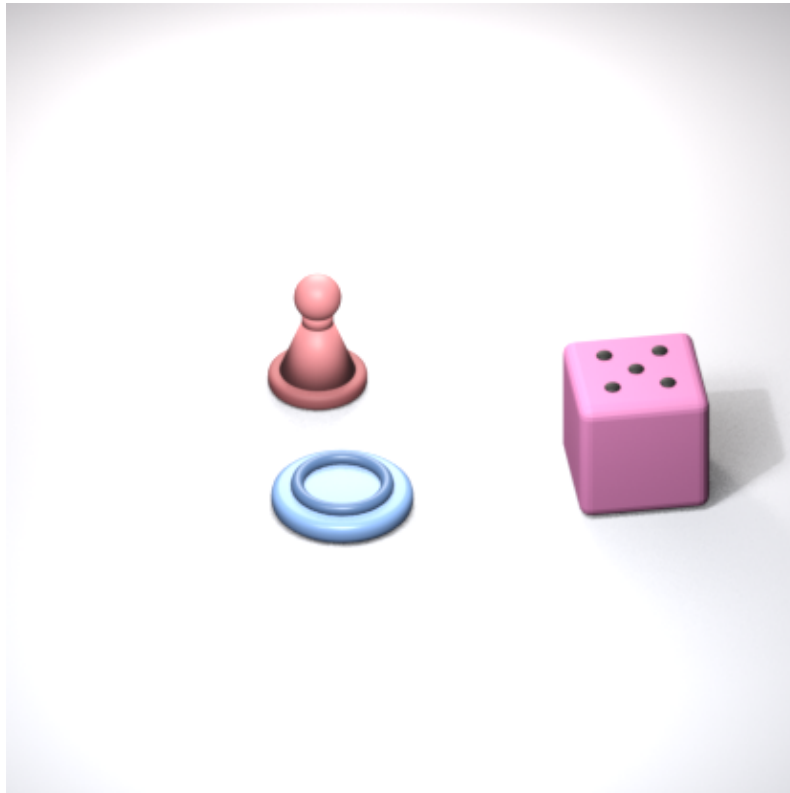


Figura 1 — Exemplo de cena sintética renderizada com os três objetos. Fonte: elaboração própria (2026).

Automação e reprodutibilidade

O arquivo `blender/generate_dataset.py` configura toda a cena, recebe por linha de comando as quantidades de treino, validação e teste, a resolução e a semente aleatória. A semente 42 permite repetir a mesma sequência de imagens. Ao final da execução, o script também salva o arquivo `boardvision_scene.blend`, além de metadados em JSON Lines com os parâmetros usados em cada cena.

4. Geração e anotação automática

Randomização da cena

Para cada renderização, os objetos recebem posições amostradas em uma área útil do plano, com distância mínima aproximada de 1,85 unidade. A rotação cobre 360 graus e a escala varia entre 0,78 e 1,16. A câmera ortográfica muda levemente de posição, altura, alvo e escala. As energias das luzes também são sorteadas, produzindo sombras e contrastes diferentes. Cores foram amostradas no espaço HSV para manter variedade sem gerar objetos excessivamente escuros.

Bounding boxes no formato YOLO

Depois de atualizar a cena, o script percorre os vértices das caixas envoltoras de todos os componentes de cada objeto. Esses pontos são transformados para coordenadas de câmera com `world_to_camera_view`. Os valores mínimos e máximos formam a caixa 2D, que é recortada aos limites da imagem e convertida para centro, largura e altura normalizados.

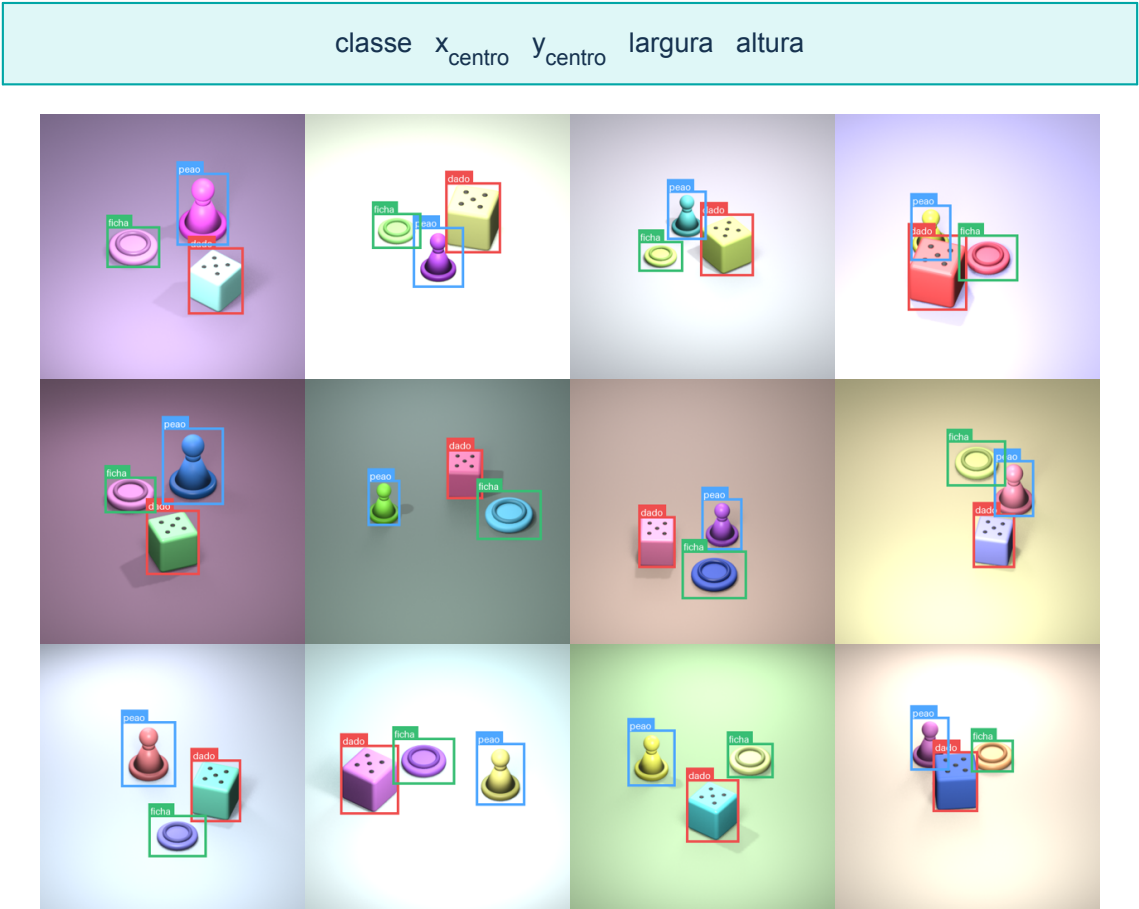


Figura 2 — Amostra das anotações geradas automaticamente. Vermelho: dado; azul: peão; verde: ficha. Fonte: elaboração própria (2026).

5. Dataset e controle de qualidade

Foram geradas 240 imagens PNG de 416×416 pixels. Como cada imagem contém um exemplar de cada classe, o conjunto possui 720 instâncias e distribuição perfeitamente equilibrada. A separação foi realizada durante a geração para manter conjuntos independentes.

Divisão	Imagens	Instâncias	Dado	Peão	Ficha	Proporção
Treino	168	504	168	168	168	70%
Validação	36	108	36	36	36	15%
Teste	36	108	36	36	36	15%
Total	240	720	240	240	240	100%

Estrutura de pastas

Caminho	Conteúdo
dataset/images/train	168 imagens para ajuste dos pesos
dataset/images/val	36 imagens para acompanhar o treinamento
dataset/images/test	36 imagens usadas somente na avaliação final
dataset/labels/{split}	um arquivo TXT YOLO para cada imagem
dataset/metadata.jsonl	parâmetros de cena e caixas por imagem
dataset/data.yaml	classes e caminhos usados pelo detector

Validação automática

O script `scripts/validate_dataset.py` verifica correspondência entre imagens e rótulos, dimensões, identificadores de classe, quantidade de campos, largura e altura positivas e coordenadas normalizadas. O resultado foi 240 pares válidos, 720 instâncias e zero erros. Além do arquivo JSON de auditoria, a grade da Figura 2 foi usada para inspeção visual das caixas.

OK — 240 imagens	OK — 240 rótulos	OK — 720 instâncias	OK — 0 erros
------------------	------------------	---------------------	--------------

6. Treinamento do modelo

Foi escolhida a arquitetura YOLOv8n, uma versão compacta adequada para experimentos rápidos de detecção. O treinamento utilizou transferência de aprendizado a partir de pesos pré-treinados. A camada de saída foi adaptada para as três classes do projeto, enquanto o restante da rede foi refinado com as imagens sintéticas.

Parâmetro	Valor
Arquitetura	YOLOv8n — 3,0 milhões de parâmetros
Épocas	20
Tamanho de entrada	416 × 416
Batch	16
Otimizador	AdamW
Taxa de aprendizado inicial	0,001
Semente	42
Dispositivo	Apple MPS (GPU do Mac)

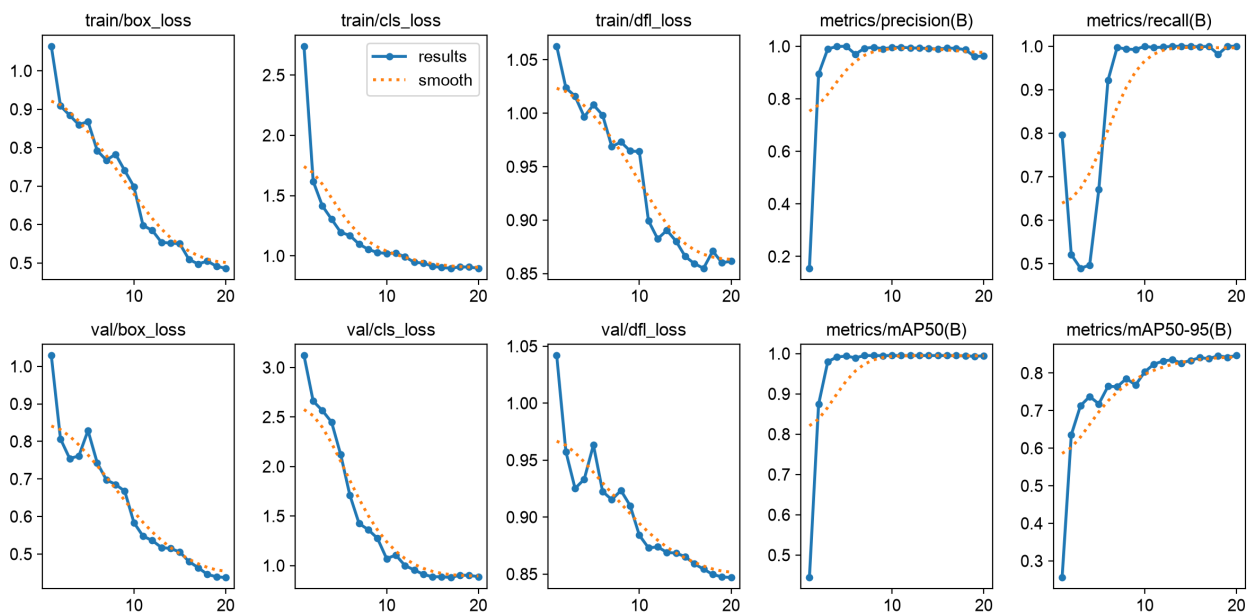


Figura 3 — Curvas de perda, precisão, recall e mAP ao longo das 20 épocas. Fonte: resultados do treinamento (2026).

As perdas de caixa, classificação e distribuição diminuíram de forma consistente. O mAP@50 subiu rapidamente e permaneceu próximo de 0,995; já o mAP@50–95 continuou melhorando ao longo das épocas, mostrando refinamento gradual da posição das caixas. O melhor peso foi salvo em models/best.pt.

7. Avaliação e resultados

A avaliação final foi realizada nas 36 imagens do conjunto de teste, contendo 108 objetos que não participaram do ajuste dos pesos nem da escolha do melhor checkpoint. As métricas globais são apresentadas a seguir.

Métrica	Resultado	Interpretação
Precisão	97,7%	poucas detecções incorretas
Recall	100,0%	todos os objetos verdadeiros foram recuperados
mAP@50	99,5%	excelente detecção no IoU mínimo de 0,50
mAP@50–95	80,9%	boa localização sob limiares mais rigorosos
Inferência	≈ 5,1 ms/imagem	medição local no Apple MPS

Resultados por classe

Classe	Precisão	Recall	mAP@50	mAP@50–95
Dado	98,9%	100,0%	99,5%	99,5%
Peão	99,5%	100,0%	99,5%	74,8%
Ficha	94,6%	100,0%	99,5%	68,3%

O dado apresentou a localização mais estável porque sua geometria produz limites retangulares bem definidos. Peões e fichas mantiveram classificação correta no IoU 0,50, mas suas formas curvas, sombras e perspectiva tornam pequenas diferenças de caixa mais relevantes nos limiares altos usados pelo mAP@50–95.

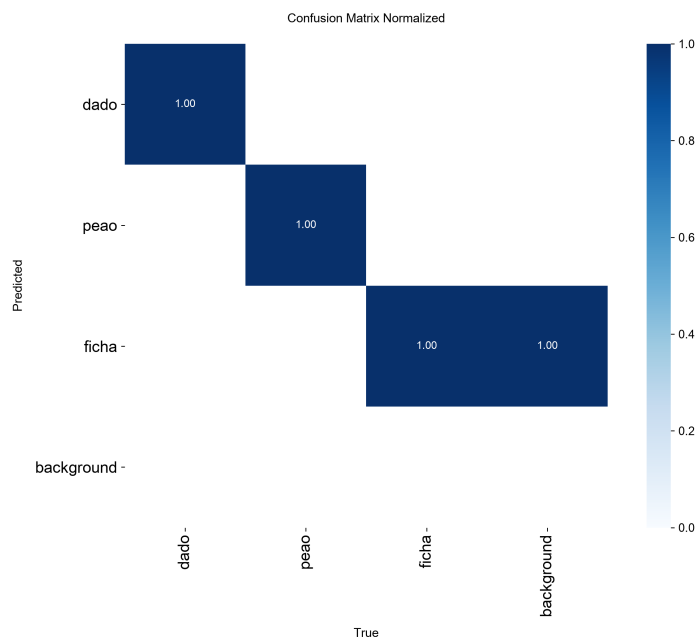


Figura 4 — Matriz de confusão normalizada no limiar de confiança 0,25. Fonte: avaliação do modelo (2026).

8. Análise qualitativa

As previsões mostram que o modelo reconhece os três objetos sob diferentes combinações de cor, posição, escala, enquadramento e iluminação. Mesmo quando as peças aparecem próximas, as caixas são separadas corretamente na maioria dos exemplos. Os valores de confiança foram mais altos para os dados e moderados para peões e fichas, comportamento coerente com os resultados por classe.



Figura 5 — Previsões do melhor modelo em 12 imagens do conjunto de teste. Fonte: elaboração própria (2026).

9. Limitações e domain gap

Os números obtidos medem o desempenho em imagens sintéticas produzidas pela mesma família de cenas. Portanto, eles não comprovam diretamente a mesma qualidade em fotografias reais. Esse intervalo entre o domínio sintético e o real é chamado domain gap.

Fotos reais podem incluir texturas, desgaste, reflexos, desfoque, lentes, fundos e oclusões que não foram simulados. Como próximos passos, o dataset pode receber texturas fotográficas, geometrias alternativas para cada classe, fundos mais complexos e oclusões controladas. Também seria importante criar um pequeno conjunto real anotado para medir o domain gap e decidir quais novas variações devem ser incluídas.

10. Instruções de uso

O repositório contém a cena do Blender, o script de geração, o dataset completo, os scripts de treinamento e avaliação, os pesos do melhor modelo, as figuras e este relatório. O README apresenta todos os comandos. O fluxo resumido é:

Etapa	Procedimento
1	Criar um ambiente Python e instalar requirements.txt.
2	Executar generate_dataset.py pelo Blender em modo background.
3	Rodar validate_dataset.py e conferir o JSON e a grade de anotações.
4	Treinar com train_yolo.py; o melhor peso será copiado para models/best.pt.
5	Avaliar com evaluate_yolo.py e consultar results/test_metrics.json.
6	Usar predict.py para processar uma imagem, vídeo ou pasta.

11. Conclusão

O BoardVision cumpriu o objetivo de integrar todas as etapas do desafio final. A cena e os objetos foram criados proceduralmente, as variações foram automatizadas, as caixas foram geradas sem anotação manual e o dataset foi validado antes do treinamento. O YOLOv8n atingiu mAP@50 de 99,5% no teste sintético, demonstrando que o conjunto contém informação suficiente para aprender as três classes propostas.

O principal aprendizado foi perceber que o valor dos dados sintéticos não está apenas em produzir muitas imagens, mas em controlar a diversidade, manter as anotações corretas, separar adequadamente os conjuntos e interpretar as métricas considerando as limitações do domínio. O projeto ficou organizado e reproduzível para permitir novos ciclos de geração e treinamento.

Referências

- BLENDER FOUNDATION. Blender Python API Documentation. Disponível em: <https://docs.blender.org/api/current/>.
- ULTRALYTICS. Object Detection Datasets Overview. Disponível em: <https://docs.ultralytics.com/datasets/detect/>.
- ULTRALYTICS. Model Training with Ultralytics YOLO. Disponível em: <https://docs.ultralytics.com/modes/train/>.
- KELLY, Adam. Tutoriais do curso sobre geração de dados sintéticos, Blender e visão computacional.

Repositório do projeto:

<https://github.com/Fefdiniz11/FastCamp-Dados-Sinteticos/tree/main/ProjetoFinal>